

Exercise 1: (8 points)

Pour résoudre le problème de reconnaissance de 100 objets, 100 images sont acquises par une caméra à différents points de vues et à différentes conditions d'illumination pour chacun des objets.

Nous prenons ensuite 10 nouvelles images de 10 objets (parmi les 100 objets) dans de nouvelles conditions (point de vue, illumination).

En utilisant l'apparence visuelle, Ecrire l'algorithme qui met en oeuvre la base de connaissances et qui permet de rechercher l'identité des objets présents dans les nouvelles images.

Algorithm 1: Dataset representation

Given M learning images $\{I_1^{(q)}, I_2^{(q)}, \dots, I_M^{(q)}\}$ for each object q (=1..Q) of Q training objects

1- Normalize all images to remove brightness variations:

$$I_m'^{(q)} = I_m^{(q)} \parallel I_m^{(q)} \parallel /$$

2- Convert image $I_m'^{(q)}$ to a vector $f_m'^{(q)}$

3- Compute the mean vector $c^{(q)}$ of each object.

4- Subtract the mean feature vector $c^{(q)}$ for object q

$$f_m^{(q)} = f_m'^{(q)} - c^{(q)}$$

5- Construct the data matrix and covariance matrix:

$$F^{(q)} = \begin{bmatrix} f_1^{(q)} & f_2^{(q)} & \dots & f_M^{(q)} \end{bmatrix}$$

$$R^{(q)} = F^{(q)} F^{(q)T}$$

6- Compute the K eigenvectors $e_1^{(q)}, e_2^{(q)}, \dots, e_K^{(q)}$ of $R^{(q)}$

7- Project feature vector to eigenvectors for **object q**:

$$p_m^{(q)} = [e_1^{(q)}, e_2^{(q)}, \dots, e_K^{(q)}]^T \times f_m^{(q)}$$

Algorithm 2: Object recognition

Given input image (I) for object recognition

1- Normalize the image to remove brightness variations:

$$I' = I \parallel I \parallel /$$

2- Convert image I' to a vector f'

For each object q in the database, perform steps 3-6:

3- Compute the mean vector $c^{(q)}$ of each object.

4- Subtract the mean feature vector $c^{(q)}$ for object q

$$f_q = f' - c^{(q)}$$

5- Project feature vector to eigenspace for object q:

$$p^q = [e_1^{(q)}, e_2^{(q)}, \dots, e_K^{(q)}]^T \times f^{(q)}$$

6- In the eigenspace of object q find the closest point to projected point, compute the distance $d^{(q)}$.

7- Find the object for which $d^{(q)}$ is minimum.

Exercice 2: (6 points) ratt 2023/2024

Nous supposons qu'une scène contenant un objet cube est observé par une caméra calibrée donnant naissance à 6 points coins (corner) localisés comme étant les intersections des arêtes sur l'image.

Une seconde prise de vue de la même scène donne naissance à une seconde image. Les points coins sont aussi localisés.

Donnez l'algorithme à suivre pour calculer la structure 3D de l'objet observé.

Solution

Pour la solution proposée par certains étudiants qui consiste à dire qu'il n'est pas possible de calculer la matrice fondamentale et puis la matrice essentielle à partir des 6 points, est considérée en correcte *partiellement*.

En effet, il existe un algorithme qui calcule la matrice essentielle à partir de 5 points.

Aussi, il est possible de trouver deux autres points en utilisant la géométrie connue de l'objet (cube).

Voir la figure 1 pour comprendre les intersections de lignes (parallèles en 3D) ce qui correspond aux projections des points à l'infini de ces droites.

Joindre les sommets correspondants pour trouver les deux points cachés (en bleu sur la figure 1).

- Calculer la matrice fondamentale, puis essentielle.
- Triangulation (ou bien rectification et triangulation)

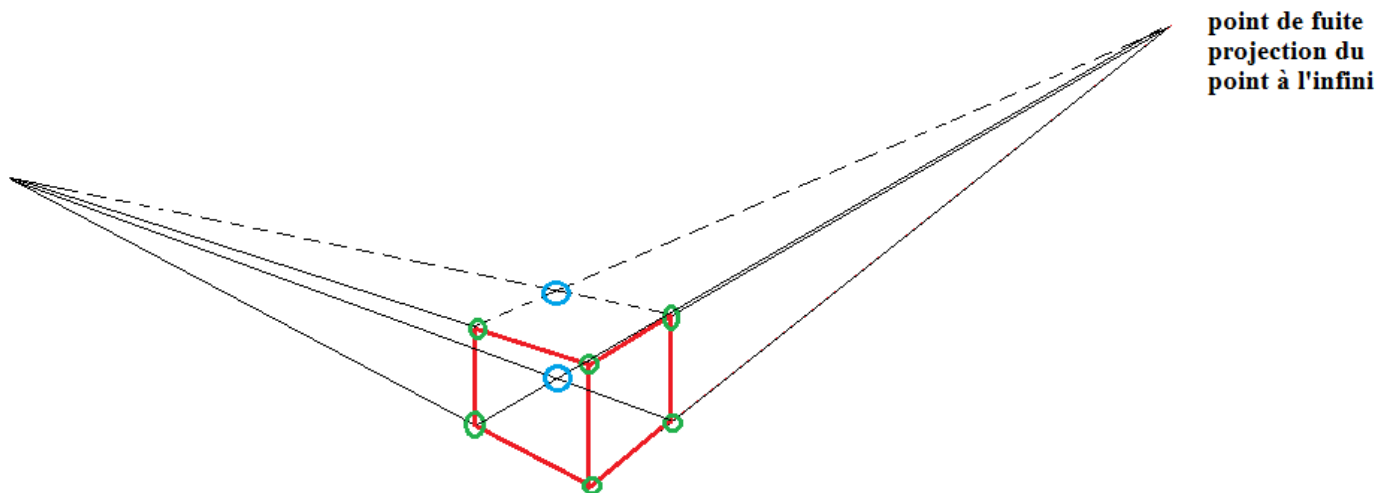


Figure 1. Calcul des points de fuite (projection des points à l'infini). On peut avoir deux ou trois.

Exercice 3: (6 points)

1- Expliquez la méthode RANSAC et discutez son utilité

RANSAC est un algorithme itératif pour estimer les paramètres d'un modèle mathématique (chercher les points planaires, alignés, etc). Il est utilisé lorsque l'ensemble de données observées peut contenir des outliers.

Son utilité : Détermination de l'homographie, matrice fondamentale, matrice essentielle, etc.

2- Donnez les étapes à suivre pour calculer la structure à partir du mouvement

1-Detect and track feature points

2-Create the centroid subtracted matrix W of corresponding feature points

3-Compute SVD of W and enforce rank constraint.

$$W=UVT=U11V1T$$

4-Set $M=U1(1)1/2Q$ and $S=Q-11)1/2V1T$

5-Find Q by enforcing the orthornormality constraint

3- Donnez les facteurs qui influent sur la précision de triangulation

- Erreurs dans la localisation des points 2D
- Erreur dans la calibration (modèle de prise de vues).

2022/2023

Exercice 2: triangle thing (solution in the video)

Exercice 1: TD Exo3

Consider the "eigenfaces" approach to face recognition in computer vision.

1. What is the role of the database population of example faces upon which this algorithm depends?

The database of example faces is the training set used to:

- Learn eigenfaces (principal components) capturing appearance variations.
- Define a low-dimensional subspace for face representation.
- Enable recognition by comparing new faces to database representations.

2. What are the features that the algorithm extracts, and how does it compute them? How is any given face represented in terms of the existing population of faces?

Features: Eigenfaces, the principal components of face images, capturing key appearance variations.

Computation:

- Normalize and vectorize face images.

- Subtract the mean face.
- Compute the covariance matrix and its eigenvectors (eigenfaces) using PCA, retaining the top k .
- Eigenfaces form a subspace.

Representation: A new face is centered, projected onto the eigenface subspace, and represented as a vector of weights $w_i = f \cdot e_i$ describing its contribution from each eigenface.

3. What are the strengths and weaknesses of this type of representation for human faces? What invariances, if any, does this algorithm capture over the factors of perspective angle (or pose), illumination geometry, and facial expression?

Strengths:

- Reduces dimensionality for efficient storage/computation.
- Captures global appearance patterns.
- Data-driven, no manual feature design.

Weaknesses:

- Sensitive to pose, illumination, and expression unless trained on diverse data.
- Misses local details, assumes linear variations.
- Requires well-aligned images.

Invariances: No inherent invariance to pose, illumination, or expression. Limited invariance possible if training data includes these variations.

4. Describe the relative computational complexity of this algorithm, its ability to learn over time, and its typical performance in face recognition trials.

Complexity:

- **Training:** Expensive ($O(N \cdot M^2 + N^3)$) due to covariance matrix and eigenvalue decomposition.
- **Recognition:** Fast ($O(k \cdot M + N \cdot k)$) for projection and comparison.
- **Storage:** Modest for eigenfaces and weights.

Learning Over Time: Not incremental; requires retraining to add new faces. Incremental PCA possible but not standard.

Performance: 80–95% accuracy in controlled settings (e.g., FERET); degrades (~50%) in unconstrained scenarios. Outperformed by modern deep learning (>99% on LFW).

Exercice 1 :

1. Quels sont les intérêts de la projection orthographique pour le calcul de la structure à partir du mouvement ?

- **Simplification mathématique** : La projection orthographique élimine la perspective, rendant les équations linéaires (pas de division par la profondeur), ce qui facilite le calcul de la structure 3D et du mouvement.
- **Stabilité numérique** : Moins sensible aux erreurs dans les petites profondeurs, contrairement à la projection perspective, évitant les singularités.
- **Adaptée aux scènes lointaines** : Convient pour des objets éloignés où la profondeur varie peu, simplifiant l'analyse dans SFM.
- **Factorisation directe** : Permet l'utilisation de la factorisation de Tomasi-Kanade pour séparer structure et mouvement.

1. What are the advantages of using orthographic projection in structure-from-motion (SfM)?

- **Simplified math**: Orthographic projection removes perspective effects, so the equations become linear (no division by depth). This makes it easier to compute the 3D structure and motion.
- **Numerical stability**: It's less sensitive to errors in depth, especially for shallow scenes, and avoids the singularities present in perspective projection.
- **Good for distant scenes**: It works well when objects are far away and depth changes are minimal, which simplifies the analysis.
- **Allows direct factorization**: Enables the use of the Tomasi-Kanade factorization method to separate motion and structure.

2. What does the observation matrix contain, and how are its elements obtained?

- **Contents**: The observation matrix $\mathbf{W}$ stores the 2D coordinates (u, v) of $\mathbf{P}$ tracked points across $\mathbf{F}$ frames. It's a matrix of size $2\mathbf{F} \times \mathbf{P}$, where:
 - Row $2f-1$ contains the **u-coordinates** of all points in frame f .
 - Row $2f$ contains the **v-coordinates** of all points in frame f .
- **How to obtain it**:
 - a. Detect and track $\mathbf{P}$ feature points (like Harris corners or SIFT features) across $\mathbf{F}$ frames using methods like optical flow or feature matching.
 - b. Center the 2D coordinates by subtracting the mean position in each frame to remove camera translation.
 - c. Fill matrix $\mathbf{W}$ with the centered (u, v) coordinates for each point and frame.

3. What is the rank of the observation matrix? Justify your answer.

- **Rank:** In the ideal case (no noise, orthographic projection), the matrix **W** has a rank of **3**.
- **Justification:**
 - The 2D point projections in **W** are generated from 3D points (structure) and camera pose (motion).
 - **W** can be written as the product $\mathbf{W} = \mathbf{M} \times \mathbf{S}$, where:
 - **M** is the motion matrix of size $(2F \times 3)$, describing the camera orientation for each frame.
 - **S** is the structure matrix of size $(3 \times P)$, containing the 3D coordinates of the tracked points.
 - The maximum rank of a matrix product is limited by the smallest number of rows or columns in the inner dimension — here, 3 — so **W** has a maximum rank of 3.
 - Noise in real data may increase the apparent rank, but theoretically it's 3.

Exercice 2 :

Une scène 3D composée de m points est projetée sur deux plans images en points images. Supposons que les deux caméras sont calibrées (seuls les matrices de calibration intrinsèque est connue), donnez la procédure à suivre pour reconstruire la scène 3D à un facteur d'échelle. (N'écrivez pas des phrases longues sans intérêt, mais des étapes avec peu de mots).

Étapes de reconstruction 3D à partir de deux images

1. **Obtenir des correspondances :**
Identifier m paires de points homologues (x_1, x_2) dans les deux images. Cela peut se faire via des descripteurs comme **SIFT** ou un suivi de caractéristiques.
2. **Normaliser les coordonnées :**
Appliquer les matrices de calibration intrinsèque inverses $\mathbf{K}_1^{-1}$ et $\mathbf{K}_2^{-1}$ aux points d'image pour obtenir des coordonnées normalisées :
 - $\tilde{x}_1 = \mathbf{K}_1^{-1} \times x_1$
 - $\tilde{x}_2 = \mathbf{K}_2^{-1} \times x_2$
3. **Estimer la matrice fondamentale :**
Calculer la matrice fondamentale **F** à partir des correspondances normalisées, par exemple en utilisant l'algorithme des 8 points avec **RANSAC**.
4. **Calculer la matrice essentielle :**
Obtenir la matrice essentielle **E** à partir de **F** et des matrices de calibration :
$$\mathbf{E} = \mathbf{K}_2^T \times \mathbf{F} \times \mathbf{K}_1$$
5. **Décomposer E :**
Extraire la rotation **R** et la translation **t** (à un facteur d'échelle près) en décomposant **E** par **SVD**, puis sélectionner la solution physique où les points sont devant les caméras.

6. Construire les matrices de projection :

- Caméra 1 : $P_1 = K_1 \times [I \mid 0]$
- Caméra 2 : $P_2 = K_2 \times [R \mid t]$

7. Trianguler les points 3D :

Pour chaque paire (x_1, x_2) , résoudre le système :

- $x_1 \sim P_1 \times X$
- $x_2 \sim P_2 \times X$

en utilisant une méthode de triangulation linéaire comme **DLT** pour obtenir les coordonnées 3D **X**.

8. Vérifier la reconstruction :

Optionnellement, ajuster les points et les poses via **bundle adjustment** pour minimiser l'erreur de reprojection.

TD Exercise 2:

Here's a **cleaned-up and fully readable version** of your answer, with **math expressions rewritten in plain text** (for copying to Word or other text editors without weird symbols):

1. Define the “Correspondence Problem,” detailing the different forms that it takes in stereovision and in motion vision.

Definition:

The **Correspondence Problem** involves identifying which points (or features) in one image correspond to the same physical point in another image — either across two views (stereovision) or over time (motion vision).

Stereovision:

- **Form:** Match features (e.g., corners, edges) between two images taken simultaneously from different viewpoints (e.g., left and right cameras).
- **Why Necessary:** Needed to triangulate 3D points from 2D projections. This enables **depth estimation** using **disparity** (differences in pixel positions across the images).

Motion Vision:

- **Form:** Match features across a sequence of images taken over time (e.g., video frames) to track the same point as it moves.
- **Why Necessary:** Required to reconstruct **3D structure** and **camera motion** through techniques like **Structure from Motion (SfM)**, by analyzing the motion of features over time.

2. Roles of Space and Time; Symmetries Between Stereo and Motion

Roles of Space and Time:

Stereovision:

- **Space:** Involves images from different spatial positions (e.g., left and right cameras). Matches are constrained by **epipolar geometry**.
- **Time:** Not relevant — both images are captured at the same moment.

Motion Vision:

- **Space:** A single camera moves over time, or multiple views are taken sequentially. Feature locations shift due to camera or object motion.
- **Time:** Essential — correspondence is tracked **across time**, from frame to frame.

Symmetries:

- Both aim to match features across views to reconstruct 3D geometry.
- **Geometric Constraints:**
 - Stereo uses **epipolar lines** and the **fundamental matrix**.
 - Motion uses **temporal coherence**, **optical flow**, and similar geometric tools.
- **Mathematical Similarity:**
Both can be modeled using **projective geometry**, differing mainly in whether the baseline is spatial (stereo) or temporal (motion).
- **Duality:**
 - **Disparity** in stereo (spatial difference) $\approx$ **motion displacement** in time
 - Both constrained by the camera model and scene geometry.

3. How does the complexity depend on the number of features (N)?

Let **N** be the number of detected features per image.

Naive Matching:

- Complexity = $O(N^2)$
Every feature in the first image is compared to every feature in the second.

With Geometric Constraints (Stereo or Motion):

- Use epipolar lines or local temporal windows to limit candidates.
- Reduces to $O(N \times M)$ where $M < N$ is the number of candidates per feature.

With Feature Descriptors (e.g., SIFT):

- Descriptor comparison adds $O(N \times D)$, where **D** is the descriptor dimensionality.

With RANSAC for Outlier Removal:

- Adds $O(I \times K)$, where:

- **I** = number of iterations
- **K** = cost per model fit (e.g., computing the fundamental matrix)

Scalability:

- While complexity grows with **N**, constraints (epipolar lines, descriptors) and robust matching (e.g., hierarchical matching, tracking) **greatly reduce the search space**.

4. General Efficient Approach to the Correspondence Problem

Feature-Based Matching with Epipolar Constraints:

1. Feature Detection:

Use robust detectors (e.g., **SIFT**, **Harris**) to find keypoints.

2. Descriptor Computation:

Compute local descriptors (e.g., SIFT vectors) to describe each feature.

3. Geometric Estimation:

- In **stereo**, estimate the **fundamental matrix** using **RANSAC**.
- In **motion**, track features and enforce smooth temporal paths.

4. Feature Matching:

- Compare descriptors between images.
- Restrict matching to **epipolar lines (stereo)** or **local search windows (motion)**.

5. Outlier Rejection and Refinement:

Use **RANSAC** to remove incorrect matches.

Optimize results using **bundle adjustment (motion)** or **triangulation (stereo)**.

Efficiency:

- Epipolar constraints and robust descriptors **significantly reduce** the search space.
- RANSAC ensures **robustness to noise/outliers**.
- Scalable to large datasets when combined with tracking or feature hierarchy.

Alternative:

Dense Matching techniques like **block matching** (stereo) or **optical flow** (e.g., Lucas-Kanade) can provide **pixel-level correspondences**, but are less robust and more computationally intensive than sparse feature matching.

Exercise 12:

For a stereo pair of cameras whose optical axes are parallel, separated by base distance b , both having focal length f , suppose a target point projects onto points in the two image planes which are outside the optical axes oppositely by amounts α and β :

1. What is the computed target depth d ?

Setup: Two cameras with parallel optical axes, separated by a baseline b , and focal length f . A target point projects to positions offset from the optical axes by α in the left image and β in the right image, in opposite directions.

Disparity:

Since the offsets are opposite, the disparity δ is calculated as:

$$\delta = |\alpha - (-\beta)| = |\alpha + \beta|$$

Depth Formula:

For parallel cameras, the depth d is computed using similar triangles:

$$d = (f \times b) / \delta = (f \times b) / |\alpha + \beta|$$

Result:

$$d = (f \times b) / |\alpha + \beta|$$

2. Why is camera calibration so important for stereo vision computations?

Accurate Geometry:

Calibration gives intrinsic parameters (like focal length f and principal point) and extrinsic parameters (like baseline b and camera rotation/translation). These are critical for converting disparity into depth accurately.

Epipolar Alignment:

Calibration ensures that epipolar lines are horizontal (in the case of rectified stereo), which simplifies the feature correspondence search.

Distortion Correction:

Lens distortion can severely affect feature matching. Calibration removes this distortion, improving accuracy.

Precision:

Small calibration errors can result in large depth estimation errors, especially for distant objects. This is because depth is **inversely proportional to disparity**, which means errors in pixel-level measurements can greatly affect depth.

3. Identify four relevant camera degrees-of-freedom and briefly explain their importance for stereo vision algorithms.

1. Focal Length (f):

- **Importance:** Affects the scale of image projection. Accurate knowledge of f is essential because depth is proportional to f (i.e., $d \propto f$).

2. **Baseline (b):**

- **Importance:** The physical distance between the two cameras (along the x-axis). A larger baseline improves depth resolution but reduces the overlapping field of view.

3. **Principal Point (cx, cy):**

- **Importance:** Indicates the optical center of the image. If incorrectly set, it leads to errors in disparity and epipolar geometry, affecting 3D reconstruction.

4. **Rotation (between cameras):**

- **Importance:** Determines the relative orientation of the two cameras. Ideally, optical axes are parallel. If not, epipolar lines become slanted, making correspondence more difficult and requiring more complex rectification.